

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

		PRÁCTICA DE LABORATORIO
CARRERA: Computación		ASIGNATURA: Computación Paralela
NRO. PRÁCTICA:	4	TÍTULO PRÁCTICA: Desarrollo e implementación de aplicaciones de computo paralelo basado en procesos e hilos
OBJETIVOS ALCANZADO: • Diseñar e implementar una aplicación concurrente que simule un sistema urbano de monitoreo ambiental. • Aplicar paralelismo basado en procesos y paralelismo basado en hilos en Python. • Analizar las diferencias de rendimiento, comunicación y sincronización entre threading y multiprocessing. • Comprender el impacto del Global Interpreter Lock (GIL) en aplicaciones paralelas basadas en hilos. • Utilizar mecanismos de sincronización y comunicación entre procesos e hilos. • Incorporar una interfaz gráfica para visualizar el funcionamiento del sistema concurrente.		
REPOSITORIO: https://github.com/KarenS22/Simulador-Ambiental.git		
ACTIVIDADES DESARROLLADAS		
1. Diseño de Modelos y Arquitectura Se definieron clases para representar los datos ambientales, con un diseño orientado a objetos. <pre> Simulador-Ambiental/ ├── core/ │ ├── base_analizador.py # Clase base con algoritmos de CPU intensivos y alertas. │ ├── base_controlador.py # Clase base con inicialización de estaciones y registro. │ └── secuencial/ # Módulo Secuencial │ ├── __init__.py │ ├── analizador.py │ └── controlador.py │ └── hilos/ # Módulo Concurrente basado en Hilos │ ├── __init__.py │ ├── analizador.py │ └── controlador.py │ └── procesos/ # Módulo Paralelo basado en Procesos │ ├── __init__.py │ ├── analizador.py │ └── controlador.py ├── models/ │ ├── alerta.py # Modelo AlertaAmbiental. │ ├── estacion.py # Modelo EstacionAmbiental (genera mediciones aleatorias). │ └── medicion.py # Modelo Medicion (almacena valores individuales). ├── gui/ │ └── ventana.py # Interfaz gráfica (Tkinter) moderna y oscura. └── main.py # Punto de entrada de la aplicación (Modo CLI y GUI). </pre>		

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

2. Implementación de Versión Secuencial (Línea Base) Se desarrolló el flujo lineal para la recolección de datos, sirviendo como punto de base para las demás versiones.

```

ControladorSecuencial.py

1  for ciclo in range(ciclos):
2      self.mtricas["ciclo_actual"] = ciclo + 1
3      self.mtricas["progreso"] = ((ciclo + 1) / ciclos) * 100
4
5      for estacion in self.estaciones:
6          self._notificar_estado(estacion.id_estacion, "activa")
7          mediciones = estacion.generar_mediciones_ciclo()
8          for medicion in mediciones:
9              self.registrar_medicion(medicion, realizar_analisis=True)
10         self._notificar_estado(estacion.id_estacion, "esperando")

```

3. Implementación de Concurrencia con Hilos (Threading) Se utilizó threading.Thread para ejecutar cada estación en un hilo separado. Se aplicó un Barrier para sincronizar los ciclos de las estaciones y un Lock para proteger el buffer compartido.

```

ControladorHilos.py

1
2  def _hilo_estacion(self, estacion, ciclos):
3      for ciclo in range(ciclos):
4          self._notificar_estado(estacion.id_estacion, "activa")
5          mediciones = estacion.generar_mediciones_ciclo()
6          self.analizador.realizar_analisis_pesado(mediciones)
7
8          for medicion in mediciones:
9              with self.lock:
10                 self.registrar_medicion(medicion, realizar_analisis=False)
11
12         self._notificar_estado(estacion.id_estacion, "esperando")

```

4. Implementación de Paralelismo con Procesos (Multiprocessing) Cada estación corre en un subproceso real del SO via multiprocessing.Process. La comunicación se realiza mediante una Queue y la sincronización de ciclos mediante un Barrier de procesos.

```

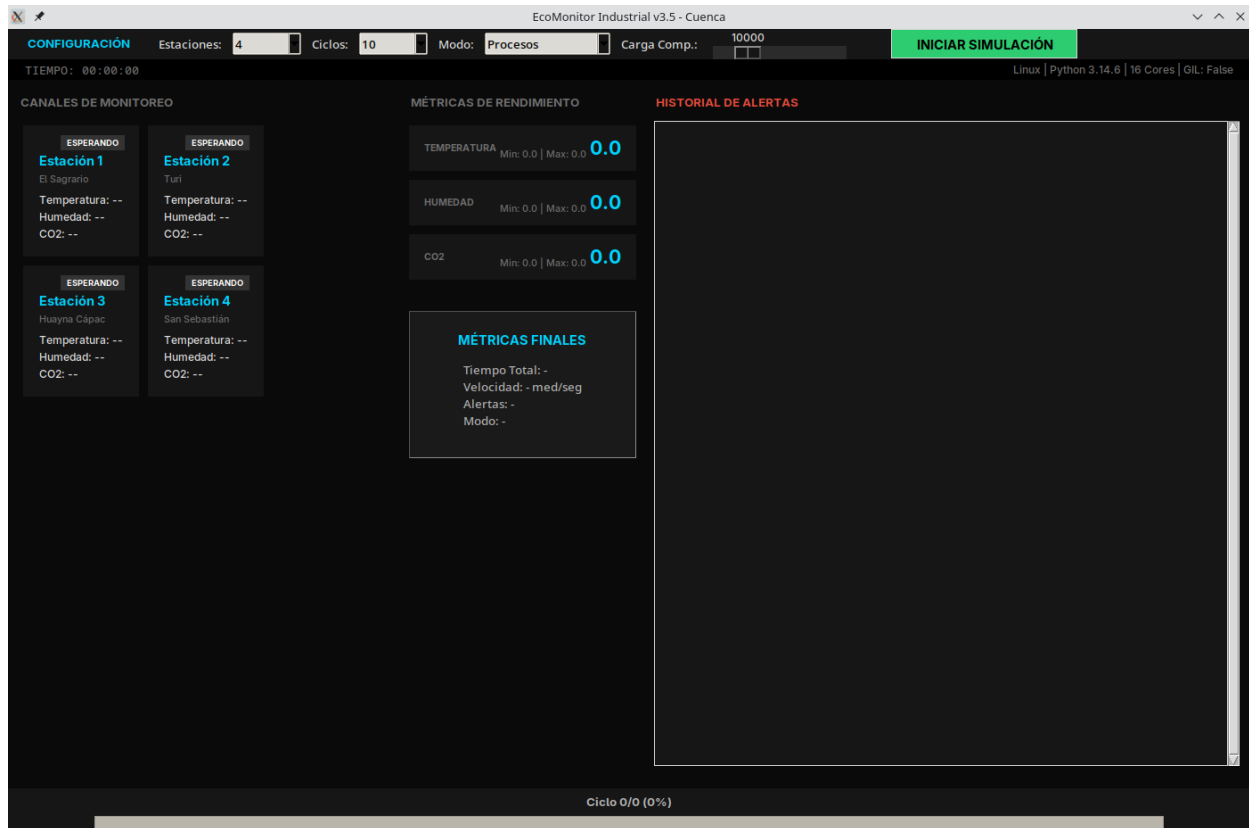
ControladorProcesos.py

1  def ejecutar(self, ciclos=10):
2      self._preparar_ejecucion("Procesos", ciclos)
3      start_time = time.perf_counter()
4
5      queue = multiprocessing.Queue()
6      barrier = multiprocessing.Barrier(len(self.estaciones))
7
8      processes = []
9      for est in self.estaciones:
10         p = multiprocessing.Process(
11             target=self._proceso_estacion,
12             args=(est, ciclos, queue, barrier, self.analizador.carga_computacional))
13         processes.append(p)
14         p.start()

```

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

5. Desarrollo de Interfaz Gráfica (Tkinter) Diseño de un dashboard para la visualización de datos en tiempo real, KPIs y historial de alertas.



6. Análisis de Rendimiento Se realizaron pruebas de carga computacional comparando los tiempos entre los tres modos

```

--- RESULTADOS FINALES ---
Estaciones: 4 Ciclos: 10 Modo: Secuencial
Tiempo Total: 8.206s
Tiempo Promedio/Ciclo: 0.821s
Velocidad: 14.6 med/seg
Alertas: 19
Zona de Riesgo: San Sebastián

--- RESULTADOS FINALES ---
Estaciones: 4 Ciclos: 10 Modo: Hilos
Tiempo Total: 0.622s
Tiempo Promedio/Ciclo: 0.062s
Velocidad: 192.9 med/seg
Alertas: 19
Zona de Riesgo: Huayna Cápac

--- RESULTADOS FINALES ---
Estaciones: 4 Ciclos: 10 Modo: Procesos
Tiempo Total: 0.774s
Tiempo Promedio/Ciclo: 0.077s
Velocidad: 155.0 med/seg
Alertas: 19
Zona de Riesgo: El Sagrario

```

RESULTADO(S) OBTENIDO(S):

Se obtuvo un simulador ambiental en Python con tres modos de ejecución: secuencial, con hilos y procesos, donde se puede comparar la velocidad de las tres versiones.

A continuación se realizaron diferentes pruebas con eslabos de simulación distintas

Tamaño	Tiempo Secuencial (Ts)	Tiempo Hilos (Th)	Tiempo Procesos (Tpr)
4 Estaciones/ 10 ciclos	4.05s	0.19s	0.28s
8 Estaciones/ 20 ciclos	17.23s	0.78s	0.86s
12 Estaciones/ 30 ciclos	39.61s	1.75s	1.97s

Se mantiene una tendencia de aceleramiento lineal en versiones concurrentes. El modelo de hilos resultó ligeramente superior al de procesos en este entorno, eliminando la sobrecarga de creación de procesos y paso de datos por Queue.

CONCLUSIONES:

La programación basada en procesos es la más eficiente para tareas con alta carga de CPU en Python, ya que utiliza núcleos físicos de forma independiente.

El uso de Python sin GIL revoluciona la programación con hilos, permitiendo que tareas matemáticas escalen de forma casi lineal, algo antes reservado solo a procesos.

RECOMENDACIONES:

Utilizar Multiprocessing cuando la tarea a realizar sea puramente matemática o de procesamiento de datos independiente ("CPU-bound").

Implementar Threading para tareas que involucren tiempos de espera (I/O) o cuando se trabaje en versiones de Python optimizadas para la eliminación del GIL.

ANÁLISIS COMPARATIVO:**1. ¿Qué versión fue más rápida?**

En todas las pruebas realizadas, la versión de hilos fue la más rápida, seguida muy de cerca por la de procesos. Esto se debe a la eficiencia del hilo nativo en Python sin GIL.

2. ¿La versión con hilos mejoró respecto a la versión secuencial?

Sí, mejoró drásticamente en todas las escalas, logrando factores de aceleración superiores a 20x.

3. ¿La versión con procesos mejoró respecto a la versión secuencial?

Sí, también mostró una mejora masiva, siendo siempre órdenes de magnitud más rápida que la secuencial.

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

4. ¿Qué impacto tuvo el GIL en la versión con hilos?

Debido a que se utilizó una versión de Python sin GIL, el impacto fue inexistente. Los hilos pudieron utilizar múltiples núcleos de CPU simultáneamente.

5. ¿Qué mecanismo de sincronización fue más importante y por qué?

El **Lock**, ya que sin él, las estaciones escribirían en las estadísticas globales de forma desordenada, causando errores de cálculo o cierres inesperados.

6. ¿Qué mecanismo de comunicación entre procesos se utilizó y por qué?

Se utilizó **multiprocessing.Queue**, porque permite el paso de objetos **Medicion** entre procesos de forma segura y evita bloqueos mutuos.

7. ¿Qué problemas de concurrencia aparecieron y cómo se resolvieron?

El principal problema fue la serialización del **Lock**. Se resolvió limitando el **Lock** exclusivamente al guardado de datos, dejando el procesamiento pesado en paralelo.

8. ¿En qué tipo de tareas conviene usar hilos?

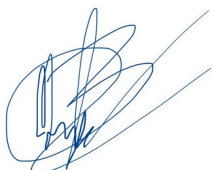
En tareas asíncronas y, en este caso particular (sin GIL), también en tareas CPU-bound para máxima velocidad con bajo consumo de RAM.

9. ¿En qué tipo de tareas conviene usar procesos?

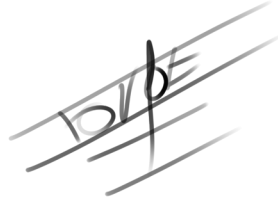
En tareas que requieran aislamiento total o cuando el costo de comunicación por colas sea pequeño comparado con la larguísima duración de la tarea.

10. ¿Qué versión resultó más compleja de implementar?

La de **procesos**, por la complejidad técnica de coordinar la cola de mensajes y la sincronización de barreras entre espacios de memoria independientes.

Nombre de estudiante:	Firma de estudiante:
Mateo Barzallo	

	VICERRECTORADO DOCENTE	Código: GUIA-PRL-001
	CONSEJO ACADÉMICO	Aprobación: 2016/04/06
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		

Jorge Cueva	
Karen Quito	